Анализ занятости парковки по камерам

Задачу выполнила соискатель
Статьева Анна Дмитриевна

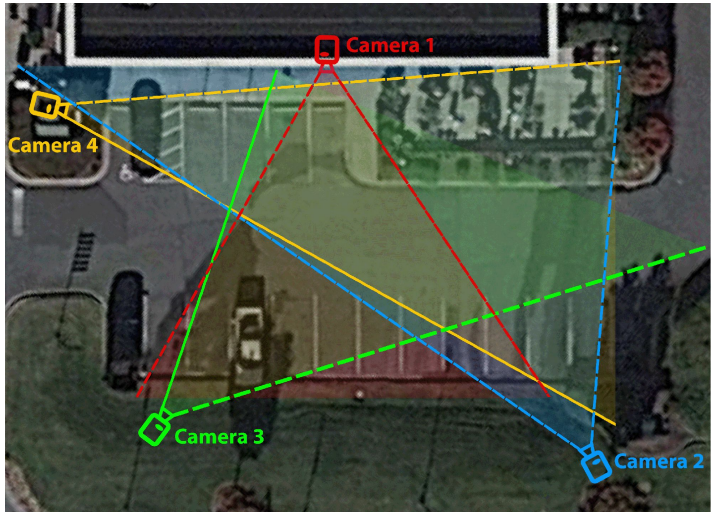
Цель работы

Необходимо сформировать в реальном времени поток событий, соответствующий актуальной занятости парковочных мест.

Посредством анализа видеопотоков с нескольких камер производить анализ занятости/доступности автомобилей на парковочных местах.

Для выполнения задания был взять датасет CHAD:

- парковка покрыта 4 камерами с перекрытием зон
- плоская уличная парковка на 14 машиномест
- Места размечены в соответствии с планом (top view) и пронумерованы



CHAD cameras' positions

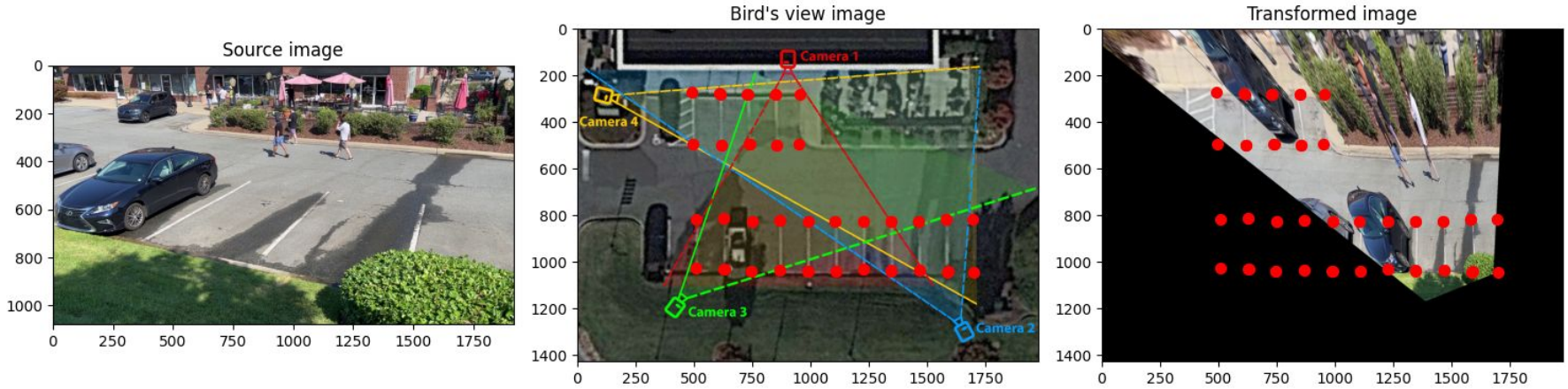
CHAD cameras' point-of-view

Основные этапы работы:

1. Развертка перспективы и соотношение мест на перспективах с top view
2. Отрисовка объемных областей для каждого места
3. Создание алгоритма определения занятого места для одной камеры
4. Реализация алгоритма занятого места совокупностью камер
5. Оптимизация алгоритма для работы в real time
6. Создание синхронных отрезков видео для демонстрации в capcut
7. Симуляция rtsp-потоков данных с .mp4 видео
8. Демонстрация работы MVP продукта

1. Развертка перспективы и соотношение мест на перспективах с top view

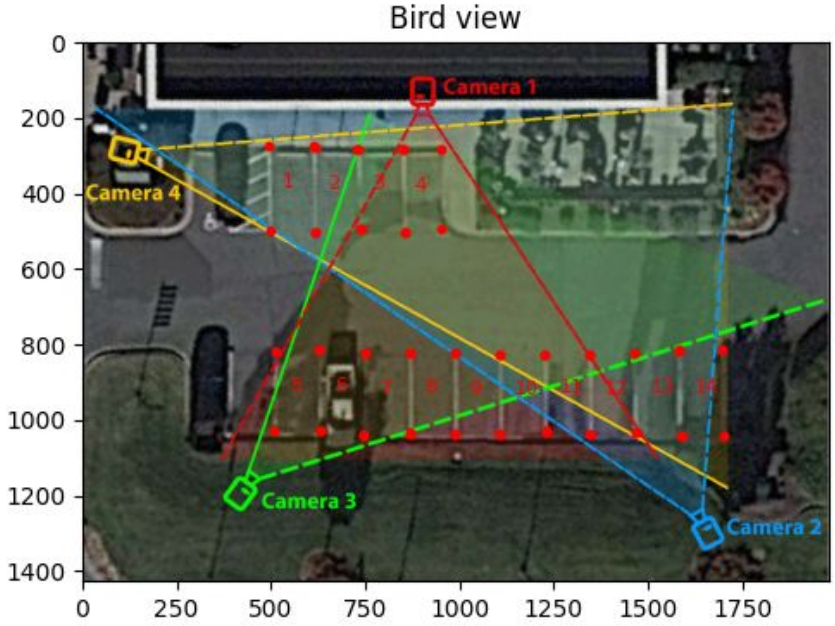
Так как параметры камер отсутствовали, отображение перспективы в вид сверху осуществлялся с помощью поиска матрицы гомографии (метод cv2.findHomography) для каждой из 4-ех камер по 4-8 точкам

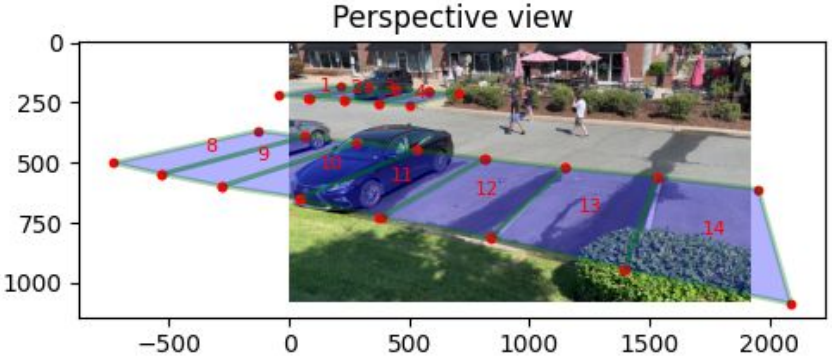


Пример отображения для камеры 2

Используемые инструменты: opencv, numpy

Пример разметки, созданной на перспективе с помощью обратной матрицы гомографии





2. Отрисовка объемных областей для каждого места

Из-за отсутствия параметров камер для отрисовки вертикальных линий была использована приблизительная формула:

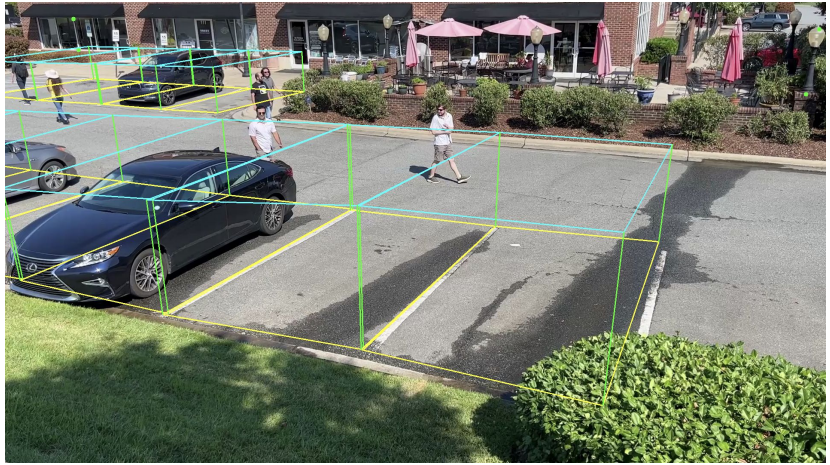
```
v = vp - p_bottom

k = C * car_height / d

p_top = p_bottom - v * k
```

`vp` - точка схода вертикалей на изображении

`p_bottom` - точка основания фигуры

`d` - расстояние от камеры до точки основания (на top view)

`C` - подбирается (по смыслу содержит внутренние параметры камеры)



Пример отрисовки для камеры 2

Длина вертикали обратно пропорциональна расстоянию до камеры, и лежит на линии соединения bottom и vp точек

Используемые инструменты: opencv, numpy

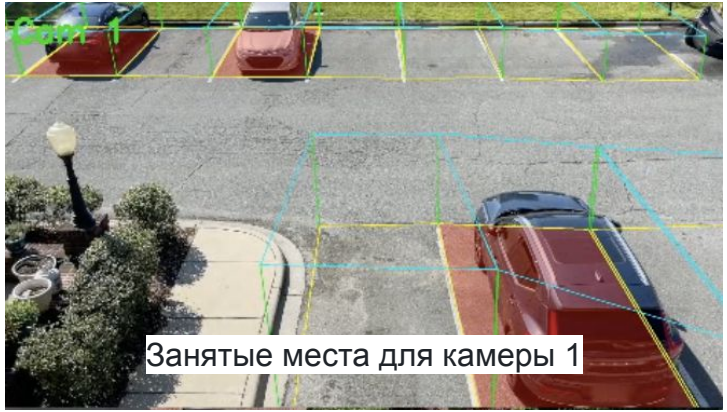
3. Создание алгоритма определения занятого места для одной камеры

В качестве алгоритма детекции машин на каждом кадре использовалась модель YOLOv8nano-seg. Считается, что машина заняла место, если:

- более 80% машины находится внутри маски объемного места
- более 20% плоского места занято этой машиной

Из всех таких мест берется место с наибольшим значением iou для этой машины

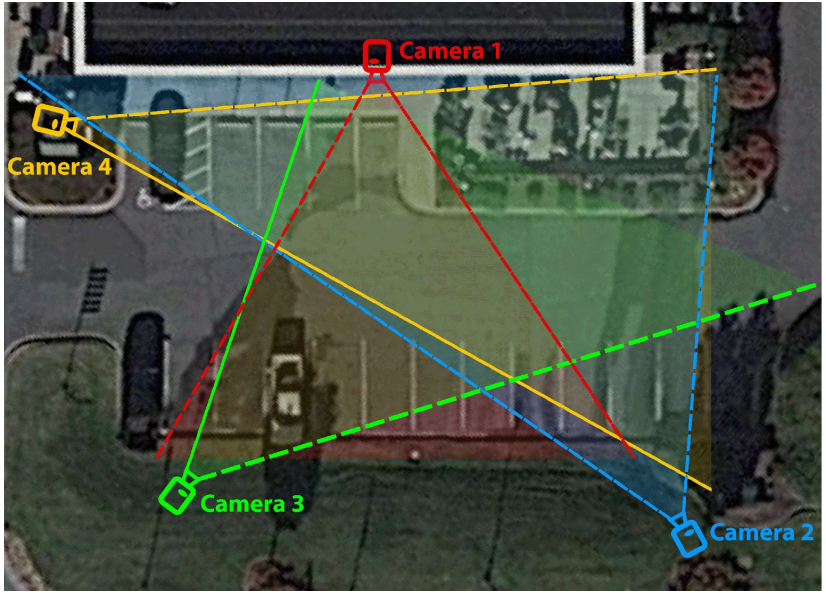
Данный алгоритм выполняется для всех найденных машин



Занятые места для камеры 1

Пример отрисовки для камеры 3

4. Реализация алгоритма поиска занятого места совокупностью камер



CHAD cameras' positions

Так как каждое место видно достаточно хорошо минимум 2-мя камерами, было принято решение:

считать место занятым, когда на 2-х и более камерах место определено как занятое

Используемые инструменты: ultrlytics, YOLO, numpy, pytorch

5. Оптимизация алгоритма для работы в real time

Для минимизации задержек обработки real time потока были приняты следующие меры:

- ускорение инференса с помощью gpu
- чтение 4-х видеопотоков отдельными процессами
- пересчет масок машин и занятых мест каждый 30-й кадр
- пересчет масок машин и занятых мест в фоновом потоке, без прерывания отображения кадров, отображение обновленных занятых мест по готовности
- расчет масок мест до начала основного пайплайна

Используемые инструменты: opencv, numpy, pytorch, cuda/mps, threading, concurrent.futures

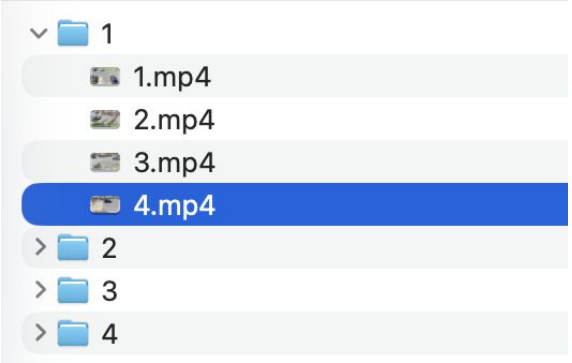
6. Создание синхронных отрезков видео для демонстрации в capcut

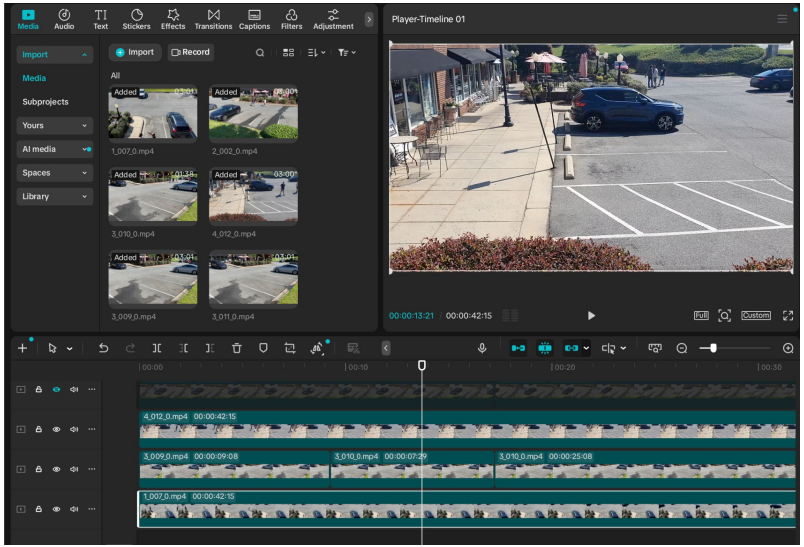
Так как видео изначально не синхронизированы, пришлось заняться самой неинтересной работой - поиск совпадающих отрывков видео

Итого: 4 синхронизированных отрезка

видео с разными ситуациями длительностью

от 1 до 8 минут





Пример работы в capcut

Используемые инструменты: capcut

7. Симуляция rtsp-потоков данных с .mp4 видео

Для симуляции работы в реальном времени был выстроен следующий пайплайн обработки синхронизированных видео:

Все 4 потока воспроизводились параллельно

video.mp4

|

FFmpeg

|

MediaMTX сервер

|

CV pipeline

Используемые инструменты: mediamtx, ffpreg

8. Демонстрация работы MVP продукта

Демонстрацию работы в режиме реального времени можно посмотреть [здесь]

Пример работы алгоритма